11.1. Introduction

This section identifies potential risks that could affect the successful completion, quality, and long-term maintainability of the Teaching Vacancies Python rewrite project. It also acknowledges existing or potential areas of technical debt that will need to be managed. Understanding these early allows for proactive mitigation and planning.

11.2. Identified Risks

11.2.1. Risk: Complexity of Data Migration

- **Description:** Migrating data from the existing Ruby on Rails PostgreSQL database to the new Python/Django PostgreSQL database, with a redesigned schema, is inherently complex. Specific challenges include:
- Mapping and transforming data between potentially different table structures and field types.
- Handling encrypted fields (e.g., PII in `User`, `JobApplication` models) which require decryption with old keys and re-encryption with new keys/libraries.
- Migrating user credentials – password hashes from Rails (likely Devise) cannot be directly used by Django's auth system; a password reset flow for users upon first login to the new system will be necessary.
- Ensuring relational integrity and correct mapping of associations (e.g., polymorphic associations, many-to-many relationships).
- Migrating data from ActiveStorage to Django's file storage solution (e.g., S3 via `django-storages`).
- **Impact:** Data loss or corruption, extended downtime during the final migration cutover, significant delays to the project timeline if migration scripts are complex and require extensive debugging.
- **Mitigation:**
- Conduct a thorough data mapping exercise between the old and new schemas before writing migration scripts.
- Develop robust ETL (Extract, Transform, Load) scripts with comprehensive logging and error handling.
- Perform multiple trial migrations on a staging environment using a recent snapshot of production data.
- Implement automated data validation checks (e.g., row counts, checksums, spot checks on critical data) to verify the integrity of migrated data.

- Clearly communicate the need for users (especially jobseekers) to reset passwords after migration.
- Plan for a well-defined cutover window with rollback procedures.

11.2.2. Risk: Ensuring 100% Feature Parity and Uncovering Edge Cases

- **Description:** The goal is feature parity with the legacy system. However, complex, long-lived applications often have undocumented features or subtle edge-case behaviors that may only be discovered late in the rewrite process or post-launch.
- **Impact:** User dissatisfaction if features they rely on are missing or behave differently; potential need for urgent post-launch fixes, impacting project timelines and team morale.
- **Mitigation:**
- Thorough analysis of the existing Rails application's codebase, routes, and user interface. (Phases 1 & 2 of this project contribute to this).
- Involve experienced users (jobseekers, publishers, support staff) in UAT and exploratory testing of the new system.
- Review existing analytics and user feedback for insights into feature usage.
- Prioritize migration of core, heavily used features first.
- Maintain a "known differences" document if some minor, low-impact features are intentionally changed or deferred.

11.2.3. Risk: Performance of the New Stack Under Production Load

- **Description:** While Python/Django is capable of handling high-traffic sites, performance issues (e.g., slow database queries, inefficient view logic, memory leaks) might arise under real-world production load that were not evident in development or staging.
- **Impact:** Poor user experience (slow page loads), increased infrastructure costs, system instability, potential for downtime.
- **Mitigation:**
- Conduct rigorous performance and load testing in a staging environment that closely mirrors production data volumes and traffic patterns.
- Implement comprehensive application performance monitoring (APM) from the outset.

- Follow best practices for Django performance (query optimization, caching strategies, efficient template rendering).
- Optimize critical paths such as vacancy search, application submission, and publisher dashboard loading.
- Ensure database connection pooling and Celery worker configurations are tuned appropriately.

11.2.4. Risk: Team Learning Curve

- **Description:** While the team is expected to have Python/Django skills, specific libraries, new patterns adopted in this project, or unfamiliar aspects of the existing system's domain logic might present a learning curve.
- **Impact:** Slower development velocity initially, potential for suboptimal implementations if complex features are tackled without full understanding.
- **Mitigation:**
- Invest in upfront training or workshops if significant skill gaps are identified.
- Promote pair programming and code reviews to share knowledge.
- Ensure clear documentation for complex or custom components.
- Allocate some buffer time in project planning for learning and ramp-up.
- Leverage well-documented, community-supported libraries where possible to reduce the need for custom solutions.

11.2.5. Risk: Integration with External Services

- **Description:** The system relies on several external services (DfE Sign-In, GOV.UK One Login, GOV.UK Notify, GIAS, DWP Find a Job, etc.). Changes in their APIs, rate limits, undocumented behavior, or outages can impact the Teaching Vacancies platform.
- **Impact:** Broken functionality (e.g., authentication, email sending, vacancy posting to DWP), delays if integration points need to be re-engineered.
- **Mitigation:**
- Develop robust client integrations with proper error handling, retries (where appropriate), and circuit breaker patterns.
- Maintain good communication channels with the teams managing these external services.

- Implement monitoring and alerting for the health and performance of integrations.
- Have contingency plans or fallback mechanisms where feasible (e.g., for email sending if Notify has issues, though this is less likely).
- Keep client libraries up-to-date.

11.2.6. Risk: Maintaining Accessibility Standards

- **Description:** Ensuring the new platform consistently meets WCAG 2.1 AA accessibility standards requires ongoing effort and attention to detail throughout development and for any future updates.
- **Impact:** Poor user experience for users with disabilities, failure to meet government digital service standards.
- **Mitigation:**
- Integrate accessibility testing (automated tools like Axe, manual checks) into the development workflow and CI/CD pipeline.
- Ensure developers and designers are trained on WCAG guidelines and GOV.UK Design System accessibility requirements.
- Conduct regular accessibility audits with specialists.
- Prioritize fixing any identified accessibility issues.

11.2.7. Risk: Security Vulnerabilities in New Code or Dependencies

- **Description:** New custom code or third-party dependencies could introduce security vulnerabilities.
- **Impact:** Data breaches, unauthorized access, system compromise, reputational damage.
- **Mitigation:**
- Follow secure coding practices (OWASP Top 10, input validation, output encoding, parameterized queries, etc.).
- Regularly update dependencies and monitor them for known vulnerabilities (e.g., using `pip-audit` or GitHub Dependabot).
- Conduct regular security code reviews.
- Perform automated static (SAST) and dynamic (DAST) security testing.
- Schedule periodic penetration tests with external security experts.
- Implement robust logging and monitoring to detect suspicious activity.

11.2.8. Risk: GeoDjango and GDAL Dependency Management

- **Description:** GeoDjango relies on the GDAL library, which can be complex to install and manage consistently across different developer machines and deployment environments. Mismatches or issues with GDAL can break migrations or runtime functionality.
- **Impact:** Delays in setting up development environments, deployment failures, runtime errors if GDAL is not correctly configured or accessible.
- **Mitigation:**
- Use containerized development environments (e.g., Docker via `.devcontainer`) to ensure consistent GDAL versions and dependencies for all developers.
- Ensure deployment images have the correct GDAL versions installed and necessary paths configured.
- Thoroughly test GIS-related functionality in staging environments that mirror production.
- Have clear documentation on setting up GDAL for local development if not fully containerized.
- The temporary use of `geopoint_placeholder` during initial model creation was a specific mitigation for early `makemigrations` issues in a restricted sandbox; this will be replaced by actual `PointField` for full functionality.

11.3. Technical Debt

- **Initial Rewrite Debt:**
- Any large-scale rewrite, especially one aiming for feature parity under time constraints, is likely to accrue some initial technical debt. This might manifest as:
- **Less-than-optimal solutions:** Some features might be implemented in a way that is functional but could be refactored for better performance, clarity, or maintainability later.
- **Incomplete test coverage for non-critical areas:** While core features will be prioritized for high test coverage, some less critical or complex edge cases might have lower coverage initially.
- **"TODO" comments or temporary workarounds:** Specific areas of code might be marked for future improvement.
- **Legacy Debt (Not Addressed):**

- The primary goal of the rewrite is to replace the legacy Ruby on Rails system, thereby eliminating its existing technical debt (e.g., outdated dependencies, hard-to-maintain code patterns). The new system does not intend to carry over known technical debt from the old platform.
- **Potential New Debt & Management:**
- **GeoDjango/GDAL Workarounds:** The temporary use of `geopoint_placeholder` (CharField) instead of `PointField` in the early model creation stages to bypass GDAL installation issues in the sandbox is a form of technical debt. This MUST be refactored to use proper 'PointField's and GeoDjango functionality once GDAL is confirmed to be working in the target deployment environments. This will require a subsequent migration.
- **Encryption Library Compatibility:** The process of finding a fully compatible and robust field-level encryption library for Django 5.x (`django-cryptography` was selected after issues with others) might have involved compromises or may require further validation. If the chosen library has limitations or if a more ideal one is identified later, this could be an area for refactoring. The temporary removal of encryption from the `User` model to pass initial migrations also constitutes debt that must be repaid by implementing a robust encryption solution.
- **SearchVectorField Updates:** The current models include `SearchVectorField` but the logic for keeping it updated (e.g., via signals or database triggers) is noted as needing refinement beyond simple `save()` method overrides. This is a known area for future improvement to ensure efficient search index updates.
- **Forward References:** The `Vacancy.publisher_ats_api_client` field uses a string forward reference to `'publishers.PublisherAtsApiClient'` because the `publishers` app and its models will be created in a later phase. This is standard Django practice but represents a dependency to be fulfilled.
- **Intention:** All identified technical debt will be tracked (e.g., in the project backlog or via code comments like `# TODO (TECH_DEBT):`). There will be a conscious effort to prioritize and pay down significant technical debt in subsequent development sprints, especially post-MVP launch, to ensure the long-term health and maintainability of the platform. Regular refactoring sessions should be part of the development lifecycle.